

Bàn về nguyên lý và thiết kế thuật toán ngẫu nhiên hóa

Zhou Yongji

Nguồn gốc: On the principles and design of randomized algorithms

IOI China candidate team papers / 1999 / Zhou Yongji.pdf

Từ khóa. Thuật toán ngẫu nhiên hóa; tính ổn định.

Tóm tắt

Bài viết này đề xuất một loại thuật toán mới để giải các bài toán tin học: thuật toán ngẫu nhiên hóa, đồng thời thảo luận nguyên lý và phương pháp thiết kế của nó. Trước hết bài viết đưa ra định nghĩa thuật toán ngẫu nhiên hóa và giải thích rằng, do ảnh hưởng của “vận may”, ta phải phân tích tính ổn định của thuật toán ngẫu nhiên hóa. Sau đó, bài viết chia thành ba trường hợp: “ngẫu nhiên không ảnh hưởng tới kết quả thực thi”, “ngẫu nhiên ảnh hưởng tới tính đúng của kết quả thực thi”, và “ngẫu nhiên ảnh hưởng tới chất lượng của kết quả thực thi”. Từ các thuật toán cơ bản đến các ví dụ dùng thuật toán ngẫu nhiên hóa để giải hiệu quả bài thi, bài viết phân tích chi tiết nguyên lý và cách thiết kế thuật toán ngẫu nhiên hóa trong cả ba trường hợp. Cuối cùng, bài viết tổng kết nguyên lý cơ bản và tính chất chung của thuật toán ngẫu nhiên hóa, nêu phương pháp tổng quát để thiết kế loại thuật toán này, đồng thời chỉ ra phạm vi áp dụng và những đặc điểm mà một thuật toán ngẫu nhiên hóa hiệu quả cần có.

1 Dẫn nhập

Trong bài viết này, ta nghiên cứu một khái niệm thuật toán mới: thuật toán ngẫu nhiên hóa. Như tên gọi, ngẫu nhiên hóa là việc dùng hàm ngẫu nhiên. Ở đây có thể lấy hàm $\text{RANDOM}(N)$ trong Borland Pascal hoặc Turbo Pascal làm ví dụ; giá trị trả về của nó là một số nguyên trong đoạn $[0, N - 1]$, và mỗi số nguyên được trả về với xác suất bằng nhau [1].

Một thuật toán chứa hàm ngẫu nhiên rất có thể [2] bị chi phối bởi yếu tố bất định. Người ta thường cho rằng một thuật toán bị yếu tố bất định chi phối chắc chắn không phải thuật toán hiệu quả. Chính cách nghĩ ấy đã khiến thuật toán ngẫu nhiên hóa bị xem nhẹ trong thời gian dài. Nhưng trong phần thảo luận sau, ta sẽ thấy điều hoàn toàn ngược lại: với một số bài toán cụ thể, thuật toán ngẫu nhiên hóa lại trở thành công cụ giải bài rất hiệu quả, đôi khi còn tốt hơn thuật toán không ngẫu nhiên hóa thông thường.

1.1 Định nghĩa thuật toán ngẫu nhiên hóa

Thuật toán ngẫu nhiên hóa là thuật toán có dùng hàm ngẫu nhiên, và giá trị trả về của hàm ngẫu nhiên trực tiếp hoặc gián tiếp ảnh hưởng tới luồng thực thi hoặc kết quả thực thi của thuật toán.

Theo định nghĩa này, không phải mọi thuật toán dùng hàm ngẫu nhiên đều được gọi là thuật toán ngẫu nhiên hóa. Chẳng hạn, một thuật toán có câu lệnh

$$i \leftarrow \text{RANDOM}(N),$$

nhưng biến i , ngoài việc được gán một giá trị ngẫu nhiên ở đây, không hề xuất hiện ở nơi nào khác. Rõ ràng, nếu thuật toán này không dùng hàm ngẫu nhiên ở chỗ khác, câu lệnh trên không thể ảnh hưởng tới luồng hoặc kết quả thực thi, nên thuật toán đó không thể gọi là thuật toán ngẫu nhiên hóa.

Mặt khác, nếu một thuật toán là thuật toán ngẫu nhiên hóa, thì luồng thực thi hoặc kết quả của nó sẽ chịu ảnh hưởng của hàm ngẫu nhiên được dùng trong đó. Theo tính chất và mức độ ảnh hưởng, ta chia thành ba trường hợp:

1. Ngẫu nhiên không ảnh hưởng tới kết quả thực thi. Khi ấy ngẫu nhiên tất yếu ảnh hưởng tới luồng thực thi, và hiệu ứng của nó thường biểu hiện qua sự dao động của hiệu suất thời gian.
2. Ngẫu nhiên ảnh hưởng tới tính đúng của kết quả thực thi. Trong trường hợp này, bài toán gốc yêu cầu tìm một nghiệm khả thi, hoặc bài toán gốc là bài toán quyết định [3]; hiệu ứng của ngẫu nhiên biểu hiện qua xác suất thu được nghiệm đúng khi thực thi.
3. Ngẫu nhiên ảnh hưởng tới chất lượng của kết quả thực thi. Khi ấy hiệu ứng của ngẫu nhiên biểu hiện qua độ sai khác giữa kết quả thực tế và nghiệm tối ưu hoặc kết quả kỳ vọng trên lý thuyết.

Trong hai trường hợp thứ hai và thứ ba, ảnh hưởng của ngẫu nhiên cũng có thể đi kèm ảnh hưởng tới luồng thực thi. Phần sau sẽ thảo luận lần lượt theo ba trường hợp này. Trước khi thảo luận, ta còn cần làm rõ một vấn đề.

1.2 Ngẫu nhiên hóa và “vận may”

Do tình hình thực thi của thuật toán ngẫu nhiên hóa bị yếu tố bất định chi phối, nên ngay cả cùng một thuật toán, với cùng một đầu vào trong nhiều lần chạy, tình hình thực thi cũng sẽ khác nhau, ít nhất là hơi khác nhau. Sự khác biệt có thể là tốc độ ra nghiệm, nghiệm đúng hay sai, nghiệm tốt hay kém, v.v. Ví dụ, một thuật toán ngẫu nhiên hóa có thể trong hai lần chạy cho nghiệm trước tốt hơn, nghiệm sau kém hơn. Vấn đề là trong đa số trường hợp, nhất là trong thi đấu, với cùng một đầu vào, chương trình chỉ được chạy một lần, và kết quả chạy được dùng để đánh giá thuật toán tốt hay kém. Như vậy ta sẽ quy lần chạy cho nghiệm kém là do vận may không tốt, và ngược lại. Tuy nhiên, thi đấu là so sánh thuật toán của ai hiệu quả hơn, chứ không phải ai may mắn hơn.

Một khi đã dùng hàm ngẫu nhiên, ta không thể thoát khỏi ảnh hưởng của vận may, nên mục tiêu là hạ ảnh hưởng ấy xuống mức thấp nhất. Nói cách khác, ta phải làm cho tình hình thực thi của thuật toán tương đối ổn định. Vì vậy trong các phân tích thuật toán tiếp theo, ta sẽ xét hiệu năng thuật toán trên bốn mặt:

1. Hiệu suất thời gian.
2. Tính đúng của nghiệm.
3. Mức độ tốt xấu của nghiệm, tức mức độ gần với nghiệm tối ưu.
4. Tính ổn định, tức mức biến động trong tình hình thực thi của thuật toán trên cùng một đầu vào. Biến động càng nhỏ thì càng ổn định. Tính ổn định của thuật toán không ngẫu nhiên hóa là 100%; tính ổn định của thuật toán ngẫu nhiên hóa thuộc khoảng (0%, 100%).

Thông thường, miễn là cài đặt chương trình của thuật toán không vượt quá giới hạn bộ nhớ, ta không cần cố ý nâng cao hiệu suất không gian, nên bỏ qua mục phân tích hiệu suất không gian. “Tính ổn định” ở mục thứ tư có thể là độ phức tạp thời gian trung bình của thuật toán, cũng có thể là xác suất nhận được nghiệm đúng sau khi chạy thuật toán, và cũng có thể là xác suất nghiệm thực tế đạt tới một mức chất lượng nào đó. “Tính ổn định” là một chỉ tiêu quan trọng để đánh giá thuật toán ngẫu nhiên hóa tốt hay kém.

2 Thuật toán ngẫu nhiên hóa có kết quả thực thi xác định

Trong mục này, ta lấy sắp xếp nhanh và phiên bản ngẫu nhiên hóa của nó làm ví dụ để thảo luận thuật toán ngẫu nhiên hóa có kết quả thực thi xác định. Theo phân tích trong phần dẫn nhập, nếu kết quả thực thi của một thuật toán ngẫu nhiên hóa là xác định, thì luồng thực thi của nó chắc chắn chịu ảnh hưởng của ngẫu nhiên, và ảnh hưởng này thường biểu hiện qua hiệu suất thời gian. Vì vậy trong phần dưới, ta bỏ qua phân tích về tính đúng và chất lượng của kết quả thực thi.

2.1 Thuật toán sắp xếp nhanh

Sắp xếp nhanh là một phương pháp sắp xếp thường dùng. Tư tưởng cơ bản của nó có tính đệ quy: chia dãy số cần sắp xếp thành hai phần, sao cho mỗi số ở phần trước không lớn hơn mỗi số ở phần sau; rồi tiếp tục chia riêng từng phần cho tới khi phần cần chia chỉ còn một số. Thuật toán có thể mô tả bằng giả mã sau.

```
QUICKSORT( $A, lo, hi$ )
1      if  $lo < hi$ 
2           $p \leftarrow \text{PARTITION}(A, lo, hi)$ 
3          QUICKSORT( $A, lo, p$ )
4          QUICKSORT( $A, p + 1, hi$ )
```

Nếu n số cần sắp xếp được lưu trong mảng A , gọi $\text{QUICKSORT}(A, 1, n)$ sẽ thu được n số theo thứ tự tăng dần. Thuật toán sắp xếp nhanh trên phụ thuộc vào thủ tục chia $\text{PARTITION}(A, lo, hi)$. Trong thời gian $\Theta(n)$, thủ tục này chia $A[lo..hi]$ thành hai phần: các phần tử không lớn hơn $x = A[lo]$, và các phần tử không nhỏ hơn $x = A[lo]$. Hai phần này lần lượt được đặt trong $A[lo..p]$ và $A[p + 1..hi]$. Trong thủ tục $\text{QUICKSORT}(A, lo, hi)$, ta gọi đệ quy $\text{QUICKSORT}()$ để tiếp tục chia $A[lo..p]$ và $A[p + 1..hi]$.

Có thể chứng minh [4] rằng trong trường hợp xấu nhất, chẳng hạn mỗi lần chia đều làm $p = lo$, độ phức tạp thời gian của sắp xếp nhanh là $\Theta(n^2)$; trong trường hợp tốt nhất, độ phức tạp thời gian là $\Theta(n \log_2 n)$. Nếu giả sử mọi hoán vị của đầu vào xuất hiện với xác suất bằng nhau, điều thực tế thường không đúng, thì độ phức tạp thời gian trung bình của thuật toán là $O(n \log_2 n)$.

2.2 Sắp xếp nhanh ngẫu nhiên hóa

Qua phân tích, ta thấy sắp xếp nhanh là phương pháp sắp xếp rất hiệu quả, với độ phức tạp thời gian trung bình $O(n \log_2 n)$. Nhưng trong trường hợp xấu nhất, độ phức tạp thời gian của nó là $\Theta(n^2)$, nên khi n lớn thì chạy rất chậm; xem bảng đối chiếu hiệu năng ở cuối mục này. Thật ra, nếu theo giả thiết ở trên rằng mọi hoán vị đầu vào xuất hiện với xác suất bằng nhau, xác suất gặp trường hợp xấu nhất nhỏ tới mức chỉ là $\Theta(1/n!)$, và hằng số ẩn trong $\Theta()$ rất nhỏ. Nhìn theo cách đó, sắp xếp nhanh vẫn rất có giá trị.

Nhưng thực tế thường không phù hợp với giả thiết này. Với một bài toán nào đó, phần lớn đầu vào ta gặp có thể là trường hợp xấu nhất hoặc gần xấu nhất. Một cách xử lý là không dùng $x = A[lo]$ để chia $A[lo..hi]$, mà dùng $x = A[hi]$, hoặc $x = A[(lo + hi) \div 2]$, hoặc một phần tử khác trong $A[lo..hi]$ để chia $A[lo..hi]$, tùy tình huống cụ thể. Nhưng cách này chưa giải quyết vấn đề, vì ta có thể gặp loại đầu vào sau: có ba lớp đầu vào, mỗi lớp xuất hiện với xác suất $1/3$, và ba lớp ấy lần lượt là trường hợp xấu nhất đối với $x = A[lo]$, $x = A[hi]$, và $x = A[(lo + hi) \div 2]$. Khi đó sắp xếp nhanh sẽ rất kém hiệu quả.

Sau khi ngẫu nhiên hóa sắp xếp nhanh, ta có thể khắc phục dạng vấn đề này. Tư tưởng của sắp xếp nhanh ngẫu nhiên hóa là: trong mỗi lần chia, chọn ngẫu nhiên một số trong $A[lo..hi]$ làm x để chia $A[lo..hi]$. Chỉ cần sửa nhẹ thuật toán gốc. Ta chỉ thêm hàm PARTITION_R , hàm này gọi thủ tục $\text{PARTITION}()$ ban đầu. Phần sửa đổi trong $\text{QUICKSORT_R}()$ so với QUICKSORT chính là việc gọi hàm chia

ngẫu nhiên.

```

PARTITION_R(A, lo, hi)
1          r ← RANDOM(hi - lo + 1) + lo
2          đổi chỗ A[lo] và A[r]
3          return PARTITION(A, lo, hi)

QUICKSORT_R(A, lo, hi)
1          if lo < hi
2              p ← PARTITION_R(A, lo, hi)
3              QUICKSORT_R(A, lo, p)
4              QUICKSORT_R(A, p + 1, hi)

```

2.3 Phân tích sắp xếp nhanh ngẫu nhiên hóa

Ngẫu nhiên hóa không thay đổi thủ tục chia của sắp xếp nhanh ban đầu, nên hiệu suất thời gian của sắp xếp nhanh ngẫu nhiên hóa vẫn phụ thuộc vào vị trí của số được chọn để chia trong mảng sau khi sắp xếp. Độ phức tạp thời gian xấu nhất, trung bình và tốt nhất vẫn lần lượt là $\Theta(n^2)$, $O(n \log_2 n)$, và $\Theta(n \log_2 n)$; chỉ có điều trường hợp xấu nhất và tốt nhất đã thay đổi. Chúng không còn do đầu vào quyết định nữa, mà do hàm ngẫu nhiên quyết định. Nói cách khác, ta không thể đưa ra một đầu vào xấu nhất để làm lần thực thi rơi vào trường hợp xấu nhất, trừ khi vận may không tốt.

Như đã nêu trong phần dẫn nhập, bây giờ ta phân tích tính ổn định của sắp xếp nhanh ngẫu nhiên hóa. Theo giả thiết mọi hoán vị xuất hiện với xác suất bằng nhau, giả thiết này không nhất thiết đúng, sắp xếp nhanh gặp trường hợp xấu nhất với khả năng $\Theta(1/n!)$. Giả sử $\text{RANDOM}(n)$ sinh ra n số với xác suất bằng nhau, giả thiết này hầu như chắc chắn đúng, thì sắp xếp nhanh ngẫu nhiên hóa cũng gặp trường hợp xấu nhất với khả năng $\Theta(1/n!)$. Nếu n đủ lớn, ta có xác suất hơn 99% để “gặp may”. Nói cách khác, thuật toán sắp xếp nhanh ngẫu nhiên hóa có tính ổn định rất cao.

Bảng sau đối chiếu hiệu năng của sắp xếp nhanh ban đầu và sắp xếp nhanh sau khi ngẫu nhiên hóa.

Mục phân tích	Thuật toán gốc	Thuật toán ngẫu nhiên hóa
Lý thuyết: xấu nhất	$\Theta(n^2)$	$\Theta(n^2)$
Lý thuyết: tốt nhất	$\Theta(n \log_2 n)$	$\Theta(n \log_2 n)$
Lý thuyết: trung bình	$O(n \log_2 n)$	$O(n \log_2 n)$
Lý thuyết: ổn định	$\Theta(1)$	$\Theta(1 - 1/n!)$
Thực chạy: đầu vào ngẫu nhiên ($n = 30000$)	0.22s	0.27s
Thực chạy: đầu vào xấu nhất ($n = 30000$)	66s	0.22s
Thực chạy: ổn định (n đủ lớn)	100%	> 99%
Nguyên nhân xấu nhất	đầu vào xấu nhất	giá trị ngẫu nhiên không tốt
Phụ thuộc thời gian vào đầu vào	phụ thuộc hoàn toàn	không phụ thuộc

Vài chú thích cho bảng trên:

1. Môi trường chạy chương trình là Pentium 100MHz, biên dịch bằng BP7.0.
2. Thời gian chạy chương trình tương ứng của thuật toán ngẫu nhiên hóa là trung bình của 1000 lần chạy.
3. Khi kiểm tra tính ổn định của thuật toán ngẫu nhiên hóa, chương trình tương ứng được chạy 1000 lần trên từng đầu vào khác nhau.
4. Mã chương trình xem trong QSORT.PAS.

2.4 Tiểu kết

Từ phân tích trên có thể thấy nguyên lý của thuật toán ngẫu nhiên hóa có kết quả thực thi xác định là: dùng hàm ngẫu nhiên để triệt tiêu toàn bộ hoặc một phần tác dụng của đầu vào xấu nhất, khiến hiệu suất thời gian của thuật toán không hoàn toàn phụ thuộc vào đầu vào tốt hay xấu.

Điều khiển đầu vào một cách thích hợp để làm kết quả thực thi tương đối ổn định là phương pháp thường dùng để thiết kế loại thuật toán ngẫu nhiên hóa này. Ví dụ, trong sắp xếp nhanh ngẫu nhiên hóa, mỗi lần ta chọn ngẫu nhiên x để chia $A[lo..hi]$. Hiệu ứng của phương pháp này tương đương với việc xáo ngẫu nhiên các số trong A trước khi sắp xếp. Một ví dụ khác là khi xây dựng cây nhị phân tìm kiếm, có thể xáo ngẫu nhiên thứ tự các khóa cần chèn trước, rồi lần lượt chèn vào cây, để thu được cây tìm kiếm nhị phân cân bằng hơn và nâng cao hiệu quả tìm kiếm khóa về sau.

3 Thuật toán ngẫu nhiên hóa có kết quả thực thi có thể lệch khỏi nghiệm đúng

Trong mục này ta thảo luận loại thuật toán ngẫu nhiên hóa thứ hai. Loại thuật toán này thậm chí có thể xuất kết quả sai, nhưng vẫn rất hiệu quả. Ta lấy thuật toán kiểm tra số nguyên tố làm ví dụ.

3.1 Thuật toán kiểm tra nguyên tố thô sơ

Với n nhỏ, ta có thể dùng “phương pháp sàng” để quyết định n có là số nguyên tố hay không. Với n lớn hơn một chút, ta có thể tìm trước mọi số nguyên tố trong đoạn $[2, \lfloor \sqrt{n} \rfloor]$, rồi dùng các số nguyên tố ấy thử chia n . Hai phương pháp này đều phải dựa vào mảng lớn; nếu n đủ lớn thì không còn thích hợp. Khi ấy ta chỉ có thể dùng $2, 3, \dots, \lfloor \sqrt{n} \rfloor$ để thử chia n . Một khi chia hết, n chắc chắn là hợp số; ngược lại là số nguyên tố. Thuật toán được mô tả như sau:

```
IsPRIME_NAIVE( $n$ )
1           for  $a \leftarrow 2$  to  $\lfloor \sqrt{n} \rfloor$ 
2           if  $a \mid n$ 
3           return FALSE
4           return TRUE
```

Khi cài đặt, ta có thể kiểm tra trước n có chẵn hay không, rồi dùng $3, 5, 7, 9, \dots$ thử chia n , để tăng tốc chương trình. Dù vậy, khi gặp số nguyên tố n khá lớn, thuật toán này vẫn tỏ ra rất chậm. Độ phức tạp thời gian xấu nhất của nó là $\Theta(n^{1/2})$.

3.2 Thuật toán kiểm tra nguyên tố ngẫu nhiên hóa

Đổi góc nhìn, từ định lý Fermat ta biết: nếu n là số nguyên tố và a không chia hết n , thì nhất định có

$$a^{n-1} \equiv 1 \pmod{n}.$$

Ta viết lại thành: nếu n là số nguyên tố, thì với $a = 1, 2, \dots, n-1$, có $a^{n-1} \equiv 1 \pmod{n}$. Vì vậy, nếu tồn tại số nguyên $a \in [1, n-1]$ sao cho $a^{n-1} \not\equiv 1 \pmod{n}$, thì n chắc chắn là hợp số. Xét thuật toán sau:

```
IsPRIME_R( $n, s$ )
1           for  $i \leftarrow 1$  to  $s$ 
2            $a \leftarrow \text{RANDOM}(n-1) + 1$ 
3           if  $a^{n-1} \not\equiv 1 \pmod{n}$ 
4           return FALSE
5           return TRUE
```

Thuật toán ngẫu nhiên chọn s giá trị a , kiểm tra $a^{n-1} \equiv 1 \pmod{n}$ có đúng hay không. Nếu tìm thấy một a làm công thức không đúng, thuật toán chắc chắn kết luận “ n là hợp số”; nếu mọi a được chọn đều làm $a^{n-1} \equiv 1 \pmod{n}$ đúng, thuật toán đưa ra giả thiết “ n là số nguyên tố”.

Thuật toán này chỉ sinh một loại lỗi: khi s giá trị a được chọn đều thỏa $a^{n-1} \equiv 1 \pmod{n}$ nhưng n thật ra là hợp số, thuật toán sẽ cho rằng chưa đủ bằng chứng n là hợp số và phán n là số nguyên tố.

Dòng 3 của giả mã cần tính $a^{n-1} \pmod{n}$. Ta chỉ cần $\lfloor \log_2(n-1) \rfloor$ vòng lặp để tính giá trị này. Đặt

$$n-1 = b_k 2^k + b_{k-1} 2^{k-1} + \dots + b_0 2^0,$$

trong đó $b_k b_{k-1} \dots b_0$ là biểu diễn nhị phân của $n-1$, nên $k = \lfloor \log_2(n-1) \rfloor$. Khi đó

$$a^{n-1} \pmod{n} = a^{(\dots((b_k) \cdot 2 + b_{k-1}) \cdot 2 + b_{k-2}) \cdot 2 + \dots + b_1) \cdot 2 + b_0} \pmod{n}.$$

Ký hiệu $\langle a \rangle = a \pmod{n}$, biểu thức trên có thể viết lại theo dạng bình phương lặp, và dùng k vòng lặp là tính được giá trị đó. Vì vậy độ phức tạp thời gian xấu nhất của thuật toán kiểm tra nguyên tố ngẫu nhiên hóa là $\Theta(s \log_2 n)$. Nếu xem s là hằng số, độ phức tạp thời gian là $\Theta(\log_2 n)$.

3.3 So sánh hai thuật toán

Tính ổn định của thuật toán kiểm tra nguyên tố ngẫu nhiên hóa phụ thuộc vào số lượng giá trị a trong $[1, n-1]$ thỏa $a^{n-1} \equiv 1 \pmod{n}$, với n là hợp số. Ký hiệu số lượng đó là $H(n)$, thì tính ổn định khi thuật toán phán định n là $(1 - H(n)/n)^s$. Trên thực tế, với đa số n , $H(n)/n$ có thể đạt 98%; với hầu hết mọi n , $H(n)/n$ không nhỏ hơn 50%; trong phạm vi 10000, số n có $H(n)/n < 50\%$ không quá 10. Ta có lý do để tin rằng chỉ cần chọn s thích hợp là có thể làm thuật toán có tính ổn định rất cao. Chẳng hạn lấy $s = 50$, thuật toán có tính ổn định ít nhất $1 - 2^{-50} > 99\%$ với hầu hết mọi n . Ngay cả khi lấy s nhỏ hơn, như $s = 5$, thuật toán cũng chưa chắc thu được kết quả sai.

Hiệu suất thời gian của thuật toán kiểm tra nguyên tố ngẫu nhiên hóa cao hơn nhiều so với thuật toán thô sơ; điều này có thể thấy từ độ phức tạp thời gian của chúng. Trong ứng dụng thực tế, lấy $s = 50$, $n = 761838257287$, là một số nguyên tố, và chạy chương trình tương ứng của hai thuật toán 100 lần, thì thuật toán trước cần 20 giây, thuật toán sau cần 102 giây [5]. Thật ra kiểm tra nguyên tố thường chỉ được gọi như một chương trình con, và số lần dùng thực tế có thể còn hơn 100 lần. Có thể thấy chênh lệch giữa hai thuật toán là rõ ràng.

3.4 Tiểu kết

Thuật toán ngẫu nhiên hóa có kết quả thực thi có thể lệch khỏi nghiệm đúng dựa trên nguyên lý sau: mức độ xấp xỉ của một thuật toán gần đúng chịu ảnh hưởng của một tham số nào đó. Khi tham số ấy lấy một giá trị đặc biệt, thuật toán có thể thu được nghiệm đúng. Vì vậy, chỉ cần chạy thuật toán nhiều lần, mỗi lần cho tham số lấy giá trị ngẫu nhiên trong một phạm vi chỉ định, thuật toán có khả năng thu được kết quả đúng.

Thông thường, ta có thể thiết kế loại thuật toán ngẫu nhiên hóa này như sau: trước hết chọn một thuật toán gần đúng, ở đây là thuật toán phán định dựa trên định lý Fermat; sau đó thêm điều khiển ngẫu nhiên vào thuật toán, ở đây là a ; cuối cùng thêm vòng lặp ngoài để chạy thuật toán gần đúng nhiều lần, ở đây là s , qua đó tạo thành thuật toán ngẫu nhiên hóa. Có thể dùng việc lặp lại thuật toán gần đúng để điều khiển tính ổn định của thuật toán, khiến nó đạt mức kỳ vọng.

4 Thuật toán ngẫu nhiên hóa có chất lượng kết quả biến đổi

Cuối cùng, ta thảo luận loại thuật toán ngẫu nhiên hóa thứ ba. Loại thuật toán này phần lớn nhằm vào các bài toán chỉ yêu cầu nghiệm tương đối tốt, ví dụ bài “Tính toán song song” trong NOI 1998.

4.1 Một số thuật toán tham lam

Nội dung đại ý của bài “Tính toán song song” là: nhập một biểu thức số học bốn phép tính gồm $+$, $-$, $\times$, $\div$, $($, $)$, và biến viết bằng chữ cái in hoa; xuất một đoạn lệnh để điều khiển một máy tính song song có hai bộ tính toán tính đúng giá trị biểu thức trong thời gian càng ngắn càng tốt. Điểm của chương trình sẽ phụ thuộc vào mức độ tốt xấu khi so sánh nghiệm tốt nhất mà chương trình tìm được với đáp án chuẩn, đáp án chuẩn chưa chắc là tối ưu.

Nếu dùng thuật toán tìm kiếm để giải bài này, mỗi lần mở rộng cây tìm kiếm phải xác định toán hạng hiện tại, toán tử và bộ tính toán, nên lượng tìm kiếm lớn kinh khủng. Vì bài toán không yêu cầu tìm nghiệm tối ưu, ta có thể dùng thuật toán tham lam để tìm nghiệm tương đối tốt. Tiêu chuẩn tham lam có nhiều cách chọn, chẳng hạn mỗi lần ưu tiên chọn toán tử tốn nhiều thời gian, hoặc mỗi lần chọn bộ tính toán rảnh sớm nhất. Hiện tại ta chưa tìm được một tiêu chuẩn tham lam áp dụng phổ biến, và khả năng tìm được tiêu chuẩn như vậy không lớn. Mặt khác, mỗi tiêu chuẩn hiện có chỉ có thể thu được kết quả tốt hơn trên một số đầu vào ở một mức độ nhất định; ta hoàn toàn có thể thiết kế riêng các đầu vào không phù hợp với từng tiêu chuẩn tham lam, khiến nó xuất kết quả không lý tưởng. Bị giới hạn bởi các điều trên, thuật toán tham lam thuần túy không thể giải hiệu quả bài “Tính toán song song”.

Một cách xử lý là: do thuật toán tham lam có hiệu suất thời gian rất cao, ta có thể thử tất cả các tiêu chuẩn tham lam trong cùng một chương trình, nhưng điều đó chắc chắn làm tăng rất mạnh “độ phức tạp lập trình”. Thuật toán tham lam ngẫu nhiên hóa dưới đây đưa ra một cách giải quyết tốt hơn.

4.2 Thuật toán tham lam ngẫu nhiên hóa

Tư tưởng cơ bản của thuật toán tham lam ngẫu nhiên hóa là: đặt mức độ tham lam $rate\%$, với $rate \in [0, 100]$; chọn một tiêu chuẩn tham lam tương đối tốt làm nền; mỗi lần cần tìm nghiệm tối ưu cục bộ, đổi thành tìm một nghiệm cục bộ tương đối tốt có mức độ tham lam theo tiêu chuẩn ấy không nhỏ hơn $rate\%$. Sửa đổi này có thể mô tả bằng giả mã sau:

```
PARTOPTIMIZE(rate)
1           for A ← nghiệm tối ưu cục bộ to nghiệm tệ nhất cục bộ
2           if RANDOM(101) ≤ rate
3           return A
4           return nghiệm tệ nhất cục bộ
```

Với bất kỳ tiêu chuẩn tham lam hiện có nào, đều tồn tại đầu vào không phù hợp với nó. Nói cách khác, tồn tại đầu vào khiến thuật toán, trong trường hợp không phải lần nào cũng chọn nghiệm tối ưu cục bộ, thu được nghiệm tốt hơn so với việc lần nào cũng chọn nghiệm tối ưu cục bộ. Thuật toán tham lam ngẫu nhiên hóa trên có thể bao phủ tình huống này. Đồng thời, khi $rate = 100$, kết quả của thuật toán tham lam ngẫu nhiên hóa chính là kết quả của thuật toán tham lam chưa sửa đổi ban đầu, nên thuật toán ngẫu nhiên hóa trên ít nhất không kém thuật toán không ngẫu nhiên hóa.

Tư tưởng trên tương đối đơn giản, nên không khó cài đặt. Hơn nữa, các thuật toán tham lam đều có hiệu suất thời gian cao, nên thời gian tiêu tốn cho nhiều lần tham lam cũng không dài. Mã chương trình xem trong PARALLEL.PAS. Trong cài đặt, ta còn thêm vài tối ưu khác, chẳng hạn các hạng tử lập chỉ tính một lần.

4.3 Hiệu năng của thuật toán tham lam ngẫu nhiên hóa

Do bài toán không có nhiều ràng buộc trên nghiệm, chủng loại và số lượng nghiệm có thể rất nhiều, nên khó phân tích hiệu năng của thuật toán tham lam ngẫu nhiên hóa trên lý thuyết. Tuy nhiên, ta có thể dùng dữ liệu kiểm thử của kỳ thi khi đó để kiểm tra thuật toán. Cách làm này vẫn có sức thuyết phục nhất định. Bảng sau đối chiếu kết quả chạy của các chương trình tương ứng với các thuật toán tham lam và đáp án chuẩn.

Tệp vào	Tham lam	Tham lam ngẫu nhiên hóa	Chuẩn	Chênh lệch
INPUT.001	14	14	14	0
INPUT.002	50	50	50	0
INPUT.003	55	55	55	0
INPUT.004	22	21	21	0
INPUT.005	53	47(95%) 46(5%)	46	+1
INPUT.006	42	23	22	+1
INPUT.007	130	100	100	0
INPUT.008	39	33	33	0
INPUT.009	3800	3700	3700	0
INPUT.010	230	210	210	0
INPUT.011	10	12	10	+2
INPUT.012	6300	6300	6300	0
INPUT.013	234	234(99%) 235(1%)	234	0
INPUT.014	3597	3474(42%) 3486(30%) 3462(24%) 3459(4%)	3498	-24
INPUT.015	412	353(52%) 354(22%) 356(16%) 357(6%) 359(3%) 360(1%)	370	-17
INPUT.016	1250	1150(98%) 1250(2%)	1150	0
INPUT.017	580	559(96%) 580(4%)	559	0
INPUT.018	3108	3108	3108	0
INPUT.019	0	0	0	0
INPUT.020	192	192	171	+21

Vài chú thích cho bảng trên:

1. Môi trường chạy chương trình là Pentium 100MHz, BP7.0.
2. Các số trong cột “thuật toán tham lam thông thường” là nghiệm tương đối tốt thu được bởi thuật toán dưới nhiều tiêu chuẩn tham lam khác nhau.
3. Chương trình tương ứng của thuật toán ngẫu nhiên hóa được chạy 100 lần để xác định kết quả chạy. Trong cột “thuật toán tham lam ngẫu nhiên hóa”, xác suất thu được kết quả đó được ghi trong ngoặc cạnh số. Nếu không có ngoặc, nghĩa là cả 100 lần chạy đều cho kết quả này.
4. Chênh lệch giữa thuật toán tham lam ngẫu nhiên hóa và đáp án chuẩn bằng nghiệm tương đối tốt do thuật toán tham lam ngẫu nhiên hóa thu được trừ đáp án chuẩn.
5. Mã chương trình xem trong PARALLEL.PAS.

Từ bảng trên, ta thấy thuật toán tham lam ngẫu nhiên hóa có thể thu được kết quả tốt ngang đáp án chuẩn trên phần lớn đầu vào, thậm chí trên một vài đầu vào, như INPUT.014 và INPUT.015, còn thu được kết quả tốt hơn đáp án chuẩn. Đồng thời, thuật toán này cũng có tính ổn định khá cao. Chú ý rằng tính ổn định ở đây là xác suất thu được nghiệm trong một phạm vi nào đó, chẳng hạn xác suất thu được kết quả tốt ngang đáp án chuẩn, hoặc xác suất thu được kết quả hơi tốt hơn đáp án chuẩn. Ngoài ra, hiệu suất

thời gian của nó tất nhiên sẽ không kém. Có thể thấy, thuật toán tham lam ngẫu nhiên hóa ở đây là một thuật toán hiệu quả để giải bài “Tính toán song song”.

4.4 Tiểu kết

Nguyên lý của thuật toán ngẫu nhiên hóa có chất lượng kết quả biến đổi về cơ bản giống mục trước: mức độ xấp xỉ của một thuật toán gần đúng chịu ảnh hưởng của một tham số nào đó; khi tham số thay đổi, kết quả thực thi của thuật toán sẽ có biến động tốt xấu. Chỉ cần chạy thuật toán nhiều lần, mỗi lần cho tham số lấy giá trị ngẫu nhiên trong phạm vi chỉ định, và cập nhật kịp thời nghiệm tốt nhất hiện tại sau mỗi lần chạy, nghiệm hiện tại sẽ không ngừng tiến gần nghiệm tối ưu lý thuyết.

Thiết kế thuật toán như vậy thường lấy thuật toán tham lam làm nền và cải tạo theo kiểu hàm `PARTOPTIMIZE()`. Thật ra, `PARTOPTIMIZE()` hoàn toàn có thể dùng như một khung thuật toán. Thuật toán bên ngoài thủ tục đó có thể mô tả bằng giả mã sau:

```
GREEDY_R()
1       $bA \leftarrow$  nghiệm tệ nhất
2      for  $rate \leftarrow 0$  to 100
3           $A \leftarrow \emptyset$ 
4          while chưa thu được nghiệm toàn cục
5               $a \leftarrow \text{PARTOPTIMIZE}(rate)$ 
6               $A \leftarrow A \cup \{a\}$ 
7              if  $A$  tốt hơn  $bA$ 
8                   $bA \leftarrow A$ 
9      return  $bA$ 
```

Từ ví dụ bài “Tính toán song song”, ta thấy khi giải những bài chỉ cần tìm nghiệm tương đối tốt, như các bài “Rết có hại”, “Ghi địa danh trên bản đồ” trong IOI 1997, hoặc “Đặt biển trạm” trong CTSC 1998, thuật toán ngẫu nhiên hóa có chất lượng kết quả biến đổi vẫn là một loại thuật toán hiệu quả.

5 Tổng kết

Sau các phân tích trên về ba trường hợp thuật toán ngẫu nhiên hóa, ta tổng kết như sau.

5.1 Nguyên lý và thiết kế thuật toán ngẫu nhiên hóa

Nguyên lý cơ bản của thuật toán ngẫu nhiên hóa là: khi trong một quyết định có nhiều lựa chọn nhưng không thể xác định lựa chọn nào là tốt, hoặc phải trả giá lớn mới xác định được lựa chọn tốt, nếu đa số lựa chọn là tốt thì chọn ngẫu nhiên một lựa chọn thường là một chiến lược hiệu quả. Thông thường, khi một thuật toán cần đưa ra nhiều quyết định, hoặc cần chạy một thuật toán nhiều lần, điểm này biểu hiện đặc biệt rõ.

Nguyên lý này khiến thuật toán ngẫu nhiên hóa có một tính chất chung: không có một đầu vào đặc biệt nào có thể làm thuật toán rơi vào trường hợp xấu nhất khi thực thi. Trường hợp xấu nhất có thể biểu hiện trong luồng thực thi, chủ yếu là hiệu suất thời gian, cũng có thể biểu hiện trong kết quả thực thi.

Theo nguyên lý này, khi thiết kế thuật toán ngẫu nhiên hóa, ta thường lấy một thuật toán nào đó làm mẫu, thêm yếu tố ngẫu nhiên, để thuật toán chọn ngẫu nhiên khi khó đưa ra quyết định. Khi thiết kế thuật toán có kết quả chịu ảnh hưởng của ngẫu nhiên, ta còn có thể chạy thuật toán nhiều lần, để thuật toán không ngừng tiến gần nghiệm đúng hoặc nghiệm tối ưu [6].

Trên đây chỉ là cách cơ bản để thiết kế thuật toán ngẫu nhiên hóa. Trong thực tế, thiết kế thuật toán ngẫu nhiên hóa không có công thức có thể rập khuôn. Ta phải phân tích từng bài toán cụ thể, nghiên cứu sâu, và thiết kế được thuật toán ngẫu nhiên hóa tốt. Đồng thời trong thiết kế, ta còn cần đặc biệt chú ý một vấn đề sau.

5.2 Phạm vi áp dụng và tính hiệu quả của thuật toán ngẫu nhiên hóa

Cuối cùng ta xét một vấn đề rất quan trọng đối với thuật toán ngẫu nhiên hóa: phạm vi áp dụng và tính hiệu quả của nó.

Ngoài bản thân mô tả bài toán, mọi yêu cầu của một bài toán đối với thuật toán còn gồm các yêu cầu như giới hạn bộ nhớ, giới hạn thời gian, giới hạn tính ổn định, v.v. Nếu một bài toán không bắt buộc thuật toán phải ổn định 100%, tức với cùng một đầu vào, không nhất thiết lần chạy nào cũng giống nhau, dĩ nhiên sự bất ổn định này không được quá lớn; đồng thời, để bù lại, bài toán lại yêu cầu thuật toán có hiệu năng tốt ở một vài mặt khác, chẳng hạn ra nghiệm nhanh, mà các hiệu năng này thuật toán không ngẫu nhiên hóa thông thường không đạt được, thì khi đó thuật toán ngẫu nhiên hóa có thể là một trong các ứng viên giải bài hiệu quả. Nếu không, thuật toán ngẫu nhiên hóa không thích hợp.

Khi một thuật toán ngẫu nhiên hóa thích hợp để giải một bài toán, có tính ổn định tương đối cao, đồng thời có biểu hiện nổi bật ở một vài mặt khác, chẳng hạn tốc độ nhanh, mã ngắn, v.v., và có thể làm tốt hơn thuật toán không ngẫu nhiên hóa thông thường, thì thuật toán ngẫu nhiên hóa ấy là một thuật toán thật sự hiệu quả.

Phụ lục

- [1] Nói nghiêm ngặt, các số do $\text{RANDOM}(N)$ trả về là số hỗn độn, không phải số ngẫu nhiên thật sự.
- [2] Ở đây không thể nói “nhất định”. Mục định nghĩa thuật toán ngẫu nhiên hóa đã giải thích điểm này.
- [3] Bài toán quyết định là bài toán yêu cầu phán định dữ liệu đầu vào, hoặc nếu bài toán không có đầu vào thì xem các điều kiện đã biết trong bài toán là đầu vào, có thỏa một điều kiện đặc biệt nào đó hay không.
- [4] Quá trình chứng minh như sau.

Ký hiệu $T(n)$ là độ phức tạp thời gian của $\text{QUICKSORT}(A, lo, hi = lo + n - 1)$. Rõ ràng $T(n)$ phụ thuộc vào vị trí của $x = A[lo]$, được dùng khi chia, trong $A[lo..hi]$ sau khi sắp xếp. Nếu giá trị trả về của $\text{PARTITION}(A, lo, hi)$ là p , thì

$$T(n) = T(p - lo + 1) + T(hi - p) + \Theta(n),$$

trong đó $\Theta(n)$ là độ phức tạp thời gian của $\text{PARTITION}(A, lo, hi)$.

Trong trường hợp xấu nhất,

$$T(n) = \max_{q=1,2,\dots,n-1} \{T(q) + T(n-q)\} + \Theta(n).$$

Có thể chứng minh $T(n) \leq cn^2$, trong đó c là hằng số, và khi mỗi lần chia đều làm $q = 1$, có $T(n) = \Theta(n^2)$. Vì vậy trong trường hợp xấu nhất, độ phức tạp thời gian của sắp xếp nhanh là $\Theta(n^2)$.

Chúng minh $T(n) \leq cn^2$ có thể dùng quy nạp toán học. Quá trình tóm tắt: từ giả thiết quy nạp,

$$\begin{aligned} T(n) &\leq \max_{q=1,2,\dots,n-1} \{cq^2 + c(n-q)^2\} + \Theta(n) \\ &= c \cdot \max_{q=1,2,\dots,n-1} \{q^2 + (n-q)^2\} + \Theta(n). \end{aligned}$$

Hàm theo q trong dấu max đạt giá trị lớn nhất khi $q = 1$ hoặc $q = n - 1$, nên

$$T(n) \leq cn^2 - 2c(n-1) + \Theta(n) \leq cn^2,$$

trong đó chọn c thích hợp để triệt tiêu $-2c(n-1) + \Theta(n)$. Chúng minh xong.

Trong trường hợp tốt nhất, mỗi lần chia đều chia A thành hai phần có kích thước bằng nhau. Khi đó

$$T(n) = 2T(n/2) + \Theta(n).$$

Do cây đệ quy hình thành trong quá trình sắp xếp có $\Theta(\log_2 n)$ tầng, và chi phí thời gian ở mỗi tầng đều là $\Theta(n)$, nên $T(n) = \Theta(n \log_2 n)$. Phân tích trên bỏ qua trường hợp $n/2$ không là số nguyên, nhưng điều đó không ảnh hưởng tới kết quả phân tích.

Ta đã giả sử mọi hoán vị đầu vào xuất hiện với xác suất bằng nhau. Với trường hợp trung bình,

$$\begin{aligned} T(n) &= \frac{1}{n} \sum_{q=1}^{n-1} (T(q) + T(n-q)) + \Theta(n) \\ &= \frac{2}{n} \sum_{q=1}^{n-1} T(q) + \Theta(n). \end{aligned}$$

Có thể chứng minh $T(n) \leq an \log_2 n + b$, trong đó a, b là hằng số, nên độ phức tạp thời gian trung bình của sắp xếp nhanh là $O(n \log_2 n)$.

Chúng minh $T(n) \leq an \log_2 n + b$ có thể dùng quy nạp toán học. Quá trình tóm tắt: từ giả thiết quy nạp,

$$\begin{aligned} T(n) &\leq \frac{2}{n} \sum_{q=1}^{n-1} (aq \log_2 q + b) + \Theta(n) \\ &= \frac{2a}{n} \sum_{q=1}^{n-1} q \log_2 q + \frac{2b(n-1)}{n} + \Theta(n). \end{aligned}$$

Với hạng thứ nhất,

$$\sum_{q=1}^{n-1} q \log_2 q = \sum_{q=1}^{\lceil n/2 \rceil - 1} q \log_2 q + \sum_{q=\lceil n/2 \rceil}^{n-1} q \log_2 q.$$

Trong tổng thứ nhất ở vế phải, $\log_2 q \leq \log_2 (n/2) = \log_2 n - 1$; trong tổng thứ hai, $\log_2 q \leq \log_2 n$. Vì vậy,

$$\begin{aligned} \sum_{q=1}^{n-1} q \log_2 q &\leq (\log_2 n - 1) \sum_{q=1}^{\lceil n/2 \rceil - 1} q + \log_2 n \sum_{q=\lceil n/2 \rceil}^{n-1} q \\ &= \log_2 n \sum_{q=1}^{n-1} q - \sum_{q=1}^{\lceil n/2 \rceil - 1} q \\ &\leq \frac{n(n-1)}{2} \log_2 n - \frac{1}{2} \left(\frac{n}{2} - 1 \right) \frac{n}{2} \\ &\leq \frac{n^2}{2} \log_2 n - \frac{n^2}{8}. \end{aligned}$$

Từ đó suy ra

$$T(n) \leq an \log_2 n + b + (\Theta(n) + b - an/4) \leq an \log_2 n + b,$$

trong đó chọn a, b thích hợp để triệt tiêu $\Theta(n) + b - an/4$. Chúng minh xong.

[5] Môi trường chạy là Pentium 100MHz, biên dịch bằng BP7.0.

[6] Nếu chạy nhiều lần thuật toán ngẫu nhiên hóa có kết quả thực thi xác định, hiệu suất thời gian có thể rất thấp, nên loại thuật toán ngẫu nhiên hóa này thường chỉ chạy một lần.

Tài liệu tham khảo

1. *Thuật toán thực dụng và thiết kế chương trình*, Ngô Văn Hổ và cộng sự, Nhà xuất bản Công nghiệp Điện tử.
2. *Tuyển tập đầy đủ về cấu trúc dữ liệu máy tính và thuật toán thực dụng*, Công ty Máy tính Hy Vọng Bắc Kinh, Mưu Nhân chủ biên.
3. *Toán tổ hợp*, Ngô Văn Hổ và cộng sự, Nhà xuất bản Công nghiệp Điện tử.
4. *Toán tổ hợp*, Lư Khai Trùng, Nhà xuất bản Đại học Thanh Hoa.
5. *Cấu trúc dữ liệu*, Thi Bá Lạc và cộng sự biên soạn, Nhà xuất bản Đại học Phục Đán.
6. *Dẫn luận thi toán trung học*, Nhà xuất bản Giáo dục Thượng Hải.